Program:

```
import math

labels = [&#39;A&#39;, &#39;D&#39;, &#39;F&#39;, &#39;G&#39;, &#39;V&#39;,
&#39;X&#39;]

# -------- Build ADFGVX Polybius Matrix using Key-1 --------

defbuild_polybius(key):

key = key.upper()

alphabet = &quot;ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789&quot;

used = []

forch in key:

ifch not in used and ch in alphabet:

used.append(ch)

forch in alphabet:

ifch not in used:

used.append(ch)

matrix = []

idx = 0

for _ in range(6):

matrix.append(used[idx:idx+6])

idx += 6

return matrix

# -------- ADFGVX Substitution --------

defadfgvx_substitution(plaintext, matrix):

plaintext = plaintext.upper().replace(&quot; &quot;, &quot;&quot;)

result = []


forch in plaintext:

found = False
```

```
for i in range(6):
for j in range(6):
if matrix[i][j] == ch:
result.append(labels[i] + labels[j])
found = True
break
if found:
break
return &quot; &quot;.join(result)
# -------- Single Columnar Transposition --------
defcolumnar_transposition(text, key):
text = text.replace(&quot; &quot;, &quot;&quot;)
key = key.upper()
cols = len(key)
rows = math.ceil(len(text) / cols)
# Pad only for matrix filling
padded = text + &#39;X&#39; * (rows * cols - len(text))

# Fill matrix row-wise
matrix = []
idx = 0
for _ in range(rows):
matrix.append(list(padded[idx:idx+cols]))
idx += cols

# Read columns in alphabetical order of key
order = sorted(range(len(key)), key=lambda k: key[k])
cipher = &quot;&quot;
```

```
for col in order:
for row in matrix:
cipher += row[col]

# Remove padding X characters
cipher = cipher.replace(&#39;X&#39;, &#39;&#39;) # Group into blocks of 3
return &quot; &quot;.join(cipher[i:i+3] for i in range(0, len(cipher), 3))

# -------- Driver Code --------
plain = input(&quot;Enter plaintext: &quot;)
key1 = input(&quot;Enter first key (Polybius key): &quot;)
key2 = input(&quot;Enter second key (Transposition key): &quot;)
polybius = build_polybius(key1)
intermediate = adfgvx_substitution(plain, polybius)
print(&quot;Intermediate Cipher Text:&quot;, intermediate)

final_cipher = columnar_transposition(intermediate, key2)
print(&quot;Final Ciphertext:&quot;, final_cipher)
```

OUTPUT:

Enter plaintext: COMPUTER

Enter first key (Polybius key): LEMON

Enter second key (Transposition key): BREAD

Intermediate Cipher Text: DD AG AF FG GF GD AD FX

Final Ciphertext: GGD DFG AFF AGA DFD

PROGRAM:

```
import math

# step 1

p = 3

q = 7

# step 2

n = p*q

print(&quot;n =&quot;, n)

# step 3

phi = (p-1)*(q-1)

# step 4

e = 2

while(e&lt;phi):

    if (math.gcd(e, phi) == 1):

        break

    else:

        e += 1

print(&quot;e =&quot;, e)

# step 5

k = 2

d = ((k*phi)+1)/e

print(&quot;d =&quot;, d)

print(f&#39;Public key: {e, n}&#39;)

print(f&#39;Private key: {d, n}&#39;)

# plain text

msg = 11

print(f&#39;Original message:{msg}&#39;)

# encryption
```

```
C = pow(msg, e)

C = math.fmod(C, n)

print(f&#39;Encrypted message: {C}&#39;)

# decryption

M = pow(C, d)

M = math.fmod(M, n)

print(f&#39;Decrypted message: {M}&#39;)
```

Output:

```
n = 21

e = 5

d = 5.0

Public key: (5, 21)

Private key: (5.0, 21)

Original message:11

Encrypted message: 2.0

Decrypted message: 11.0
```

Program :

```
#include<stdio.h>
long long power(long longbase,long long exp,long long mod)
{
long long result=1;
while(exp>0)
{
result=(result*base)%mod;
exp--;
}
return result;
}
int main()
{
long long P,G;
long longa,b;
long long A,B;
long longkeyA,keyB;
printf("Enter Prime Number (P): ");
scanf("%lld",&P);

printf("Enter Primitive Root (G): ");
scanf("%lld",&G);

printf("Enter Alice Private Key (a): ");
scanf("%lld",&a);

printf("Enter Bob Private Key (b): ");
scanf("%lld",&b);

A=power(G,a,P);
B=power(G,b,P);

printf("\nAlice Public Key: %lld",A);
printf("\nBob Public Key: %lld",B);

keyA=power(B,a,P);
keyB=power(A,b,P);

printf("\nShared Key Computed by Alice: %lld",keyA);
printf("\nShared Key Computed by Bob: %lld",keyB);

return 0;
}
```

<u>Output</u> :

```
Enter Prime Number (P): 2
Enter Primitive Root (G): 3
Enter Alice Private Key (a): 5
Enter Bob Private Key (b): 4

Alice Public Key: 1
Bob Public Key: 1

Shared Key Computed by Alice: 1
Shared Key Computed by Bob: 1
```